SPEC RAG Cookbook: Syntax, Patterns, Pitfalls & Many Examples

Purpose: This cookbook is optimized for Retrieval-Augmented Generation (RAG). It contains canonical SPEC syntax, high-signal examples, and explicit WRONG→RIGHT contrasts based on observed model mistakes.

1. Golden Rules (High Priority Signals for RAG)

- Define macros with: def name ' ... ' and close with a single apostrophe ('). No enddef, no code fences.
- SPEC uses C-style control flow: for (i = 1; i <= N; i++) { ... } and if (cond) { ... } else { ... }
- Use documented commands only (e.g., setor0, setor1, calcG, ascan, dscan, mesh, mv, umv, inc, ct).
- Do NOT use internal variables or undocumented symbols (e.g., g_uXX..., A[_m1], _s1, _d1, NPTS in place of a loop index).
- No Python/Batch/Shell syntax: no colons after def, no for /l, no endfor, no : indentation, no () inline math without assignment.
- One command per line preferred; semicolons are unnecessary.
- Use variables for arithmetic, then pass variables to commands (e.g., step = 0.1; mv th step).
- When unsure or unsupported by excerpts, output exactly: print "insufficient evidence".

2. Macro Definition Patterns

RIGHT (canonical):

```
def hello '
    print "Hello from SPEC"
'
```

WRONG (common mistakes):

```
# Python-style / invalid
def hello():
    print("Hello")
enddef

# Mixed markdown / fences
def hello '
    print "Hello"
'
```

```

# 3. Loops & Conditionals

## RIGHT:

```
def loop_example '
    for (i = 1; i <= 5; i++) {
        print i
    }
    if (i > 5) {
        print "done"
    } else {
        print "not yet"
    }
'
```

## WRONG:

```
def loop_example '
    for /l i in (1,1,5) do (
        echo %i%
    )
'
```

4. Motor Moves: Absolute vs Relative vs Increment

RIGHT:

```
# Absolute move
def move_abs '
    mv th 45
'

# Relative move (add delta to current)
def move_rel '
    umv th 2
'

# Increment motor by step
def move_inc '
    inc th 1
'
```

WRONG (mixing languages or inline math without assignment):

```
def move_bad '
    mv th (10 * 2)      # Inline () math not accepted; assign first
    umv th i += 1       # Python/C style increment not valid
'
```

Use variables for arithmetic:
```

```
def move_math '
    step = 0.5
    pos  = 10 * step
    mv th pos
'
```

5. Scans (ascan, dscan, mesh)

Absolute scan (ascan): ascan motor start finish intervals time

```
def scan_th '
    ascan th -2 2 40 1
'
```

Relative scan (dscan): dscan motor start finish intervals time

```
def scan_phi '
    dscan phi -5 5 10 2
'
```

Mesh (2D scan): mesh m1 s1 f1 n1 m2 s2 f2 n2 time

```
def mesh_scan '
    mesh tth -1 1 20 chi -1 1 20 1
'
```

WRONG (observed mistakes):

```
def scan_bad '
    ascan th "-2 2 40 1"   # quotes are wrong
'

def mesh_bad '
    # manual loops + ascan two motors — wrong; use mesh
    for (i = 1; i <= 20; i++) {
        for (j = 1; j <= 20; j++) {
            mv tth -1 + (i - 1) * 0.1
            mv chi -1 + (j - 1) * 0.1
            ascan th tth chi -1 1 1 1   # ascan scans only ONE motor
        }
    }
'
```

6. Orientation & Reflections (UB Matrix)

Setting reflections and updating geometry:

```
def set_reflections '
    setor0 2 0 0
    setor1 2 2 0
    calcG
'
```

Rotation in equal steps with orientation update:

```
def rotate_sample '
    setor0 0
    calcG
    for (i = 1; i <= 10; i++) {
        inc or0 36
        calcG
    }
'
```

WRONG (observed): using internal variables and pseudo constructs:

```
def bad_rotate '
    g_u03or0 10           # internal var, not a user command
    loop_through_reflex_file "reflex.dat" { ... }  # not a SPEC construct
    endfor                 # not SPEC
'
```

7. Counting & Detector Data

Counting and reading detector scaler:

```
def measure_once '
    ct 1
    print "Detector:", S[DET]
'
```

Normalize by monitor counter (site-specific mnemonic):

```
def count_norm '
    ct 1
    norm = S[DET]/S[MON]
    print "Norm:", norm
'
```

8. File I/O (open/fprintf/close)

RIGHT (SPEC file descriptors):

```
def repeat_measurement '
    local f
    f = open("repeat.dat", "w")
    for (i = 1; i <= 5; i++) {
        ct 1
        fprintf(f, "Iteration %d: Counts = %g\n", i, S[DET])
    }
    close(f)
'
```

WRONG (observed):

```
def repeat_measurement '
    open file "repeat.dat" w
    writeline file "repeat.dat", result
    close file "repeat.dat"
'
```

9. Macro Arguments & Usage

Pattern: $# (argc), $1, $2 ... (argv). Guard with usage.

```
def scan_arg '
    if ($# != 4) {
        print "Usage: scan_arg motor start finish time"
        exit
    }
    ascan $1 $2 $3 40 $4
'
```

10. Safety & Sanity Patterns

```
def safe_move '
    if (th > 180 || th < -180) {
        print "Angle out of bounds"
        exit
    }
    mv th 0
'
```

Abort & resync hints:

```
# CTRL-C aborts a motion or count
# If controller and SPEC are out of sync:
#   sync
```

11. Common Pitfalls (Observed Errors & Fixes)

■ Using internal variables (g_uXX..., A[_m1], _s1, _d1) → ■ Use documented commands (setor0
■ Python/batch loops (for /l, endfor) → ■ C-style loops only.
■ Inline arithmetic in command args → ■ Compute in variables first, then pass.
■ Quoting whole arg lists (ascan "…") → ■ No quotes for numbers/args.
■ Passing args on def line → ■ Arguments are accessed as $1, $2… inside the body.
■ Adding reflex_end without reflex_beg context → ■ Only use when actually managing a reflec
■ Using FOURC> prompt text in macros → ■ Don’t include prompt; macros contain commands only

12. Extended Examples

A) Rotate sample N steps by angle Δ (parameterized):

```
def rotate_steps '
    if ($# != 2) {
        print "Usage: rotate_steps N delta_deg"
```

```
        exit
    }
    N = $1; d = $2
    for (i = 1; i <= N; i++) {
        inc or0 d
        calcG
    }
'
```

B) Absolute scan with safeguards:

```
def safe_ascan '
    if ($# != 4) {
        print "Usage: safe_ascan motor start finish time"
        exit
    }
    ascan $1 $2 $3 40 $4
'
```

C) Mesh then count at each node:

```
def mesh_count '
    if ($# != 9) {
        print "Usage: mesh_count m1 s1 f1 n1 m2 s2 f2 n2 t"
        exit
    }
    mesh $1 $2 $3 $4 $5 $6 $7 $8 $9
'
```

D) Build reflex file and refine UB:

```
def build_reflex '
    local f
    f = open("reflex.dat", "w")
    fprintf(f, "1 0 0\n")
    fprintf(f, "0 1 0\n")
    close(f)
    reflex_beg
    # (site-specific: load reflex.dat if supported by your setup)
    reflex_end
    calcG
'
```

Appendix: Quick Reference of Frequently Used Commands

```
def, print, mv, umv, inc, ct, ascan, dscan, mesh, setor0, setor1, calcG, reflex_beg, reflex,
```